Pipeline Hazard Simulator

CS2011: Foundations of Computer Systems – Lab Assignment II

Plaksha University

Team Members:

- Niksh Hiremath (U20240158)
- Parthiv Sai Sardena (U20240070)
- Phani Srivatsav (U20240118)

---

1. Project Overview

The Pipeline Hazard Simulator is a browser-based interactive applet that lets users observe how MIPS-style instructions travel through a processor pipeline cycle by cycle. Users enter a short assembly program, choose between a 4-stage or 5-stage pipeline, and watch how data hazards are detected and resolved - either through stall insertion or optional data forwarding.

Field	Details
Live URL	https://pipeline-hazard-simulator.vercel.app/
Technology	Nextjs + TypeScript, deployed on Vercel
Pipeline Configs	4-Stage (IF, ID, EX, MEM/WB) · 5-Stage (IF, ID, EX, MEM, WB)
Hazard Support	RAW detection via stall insertion + optional data forwarding
Max Instructions	10 instructions per simulation run

---

2. Design & Architecture

The applet is a single-page Nextjs application written in TypeScript. All simulation logic is pure client-side - no server is involved. The codebase is divided into three layers:

- **UI Layer** - Nextjs components styled with Tailwind CSS
- **Simulation Engine** - hazard detection, stall insertion, forwarding resolution
- **Parser** - instruction validation and tokenisation

2.1 Pipeline Stages

Configuration	Stages	MEM/WB Handling
4-Stage	IF → ID → EX → MEM/WB	Memory access and write-back merged into one stage
5-Stage	IF → ID → EX → MEM → WB	Separate MEM and WB stages (standard textbook model)

2.2 Instruction Format & Supported Instructions

Instruction	Syntax Example	Description
ADD	`ADD R1, R2, R3`	R1 ← R2 + R3
SUB	`SUB R4, R1, R5`	R4 ← R1 − R5

Instruction	Syntax Example	Description
LW	`LW R1, 0(R2)`	R1 ← Memory[R2 + 0]
SW	`SW R1, 4(R2)`	Memory[R2 + 4] ← R1

2.3 Assumptions

- Register results become available only after the **WB stage** completes, unless data forwarding is enabled, in which case it is available only after the **EX stage** completes (or **MEM stage** for LW/SW instructions).
- At most **one instruction** enters the pipeline per cycle (single-issue, in-order).
- Only **RAW (Read-After-Write)** hazards are modelled. WAR, WAW, control, and structural hazards are out of scope.
- SW instructions have no destination register and do not produce RAW hazards.
- Forwarding paths implemented: **EX→EX** and **MEM→EX** in the 5-stage pipeline. A load-use (LW followed immediately by a dependent instruction) still requires 1 unavoidable stall even with forwarding.

---

3. Hazard Detection & Resolution Logic

3.1 RAW Hazard Detection

For every producer instruction **Ii** and consumer instruction **Ij** (j > i), the engine checks whether any source register of Ij matches the destination register of Ii. If a match is found, the engine computes how many cycles until the result is available and how many cycles Ij needs the value. The difference determines the number of stalls to insert.

3.2 Stall Insertion (No Forwarding)

Without forwarding, the result of Ii is only safe to read after Ii has completed WB. The simulator inserts **STALL** bubbles into the pipeline table. The Hazard Detection Unit indicator turns red and every hazard pair is listed in the Hazard Analysis panel.

3.3 Data Forwarding

When forwarding is enabled the Forwarding Unit becomes active. Results are forwarded:

- **EX→EX**: from the EX/MEM pipeline register back to the EX stage input
- **MEM→EX**: from the MEM/WB pipeline register back to the EX stage input

This eliminates most stalls. The single unavoidable stall is the **load-use** case - a LW result is not out of MEM in time for the very next instruction's EX stage. The pipeline table shows `EX→` and `ID<-` annotations to make forwarding paths visible.

3.4 Cycle-by-Cycle Execution

Control	Action
Next Cycle	Advance one cycle at a time
Auto Run	Run all cycles automatically to completion
Reset	Clear the simulation and start over
Cycle Counter	Top-right displays current cycle / total cycles

---

4. Test Cases

4.1 Test Case A – 5-Stage Pipeline, Forwarding Disabled

Instructions:

```
I1: lw  R1, 0(R2)
I2: add R3, R1, R4
I3: sub R5, R3, R6
I4: lw  R7, 4(R2)
```

Two chained RAW hazards: I1 produces R1 which I2 reads (load-use, 3 stalls), and I2 produces R3 which I3 reads (3 stalls). I4 is independent.

Pipeline Execution Table:

INSTR	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
I1 (LW)	IF	ID	EX	MEM	WB									
I2 (ADD)		IF	STALL	STALL	STALL	ID	EX	MEM	WB					
I3 (SUB)			IF	STALL	STALL	STALL	IF	STALL	STALL	STALL	ID	EX	MEM	WB
I4 (LW)								IF	ID	EX	MEM	WB		

Total Cycles: 14

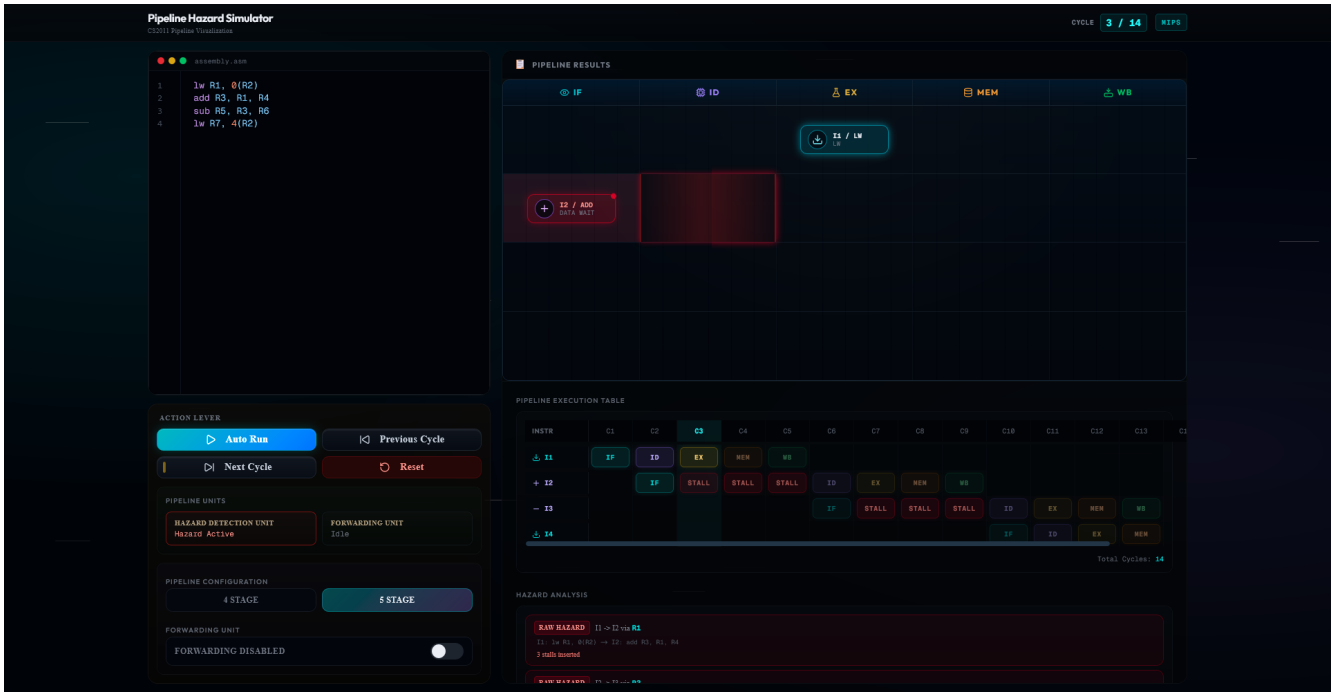

Figure 1 – 5-Stage pipeline, forwarding disabled (Cycle 3/14). I2 stalled waiting for R1 from I1. Red glow indicates DATA WAIT.

4.2 Test Case B – 5-Stage Pipeline, Forwarding Enabled

Same instruction sequence as Test Case A.

With forwarding active, EX→EX paths resolve most dependencies. The load-use hazard (I1→I2) still requires **1 stall**. The I2→I3 dependency is resolved via EX→EX forwarding with no stall.

Pipeline Execution Table:

INSTR	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1 (LW)	IF	ID	EX	MEM	WB				
I2 (ADD)		IF	STALL	ID	EX→	MEM	WB		
I3 (SUB)			IF	IF	ID←	EX←	MEM	WB	
I4 (LW)					IF	ID	EX	MEM	WB

Total Cycles: 9 (down from 14)

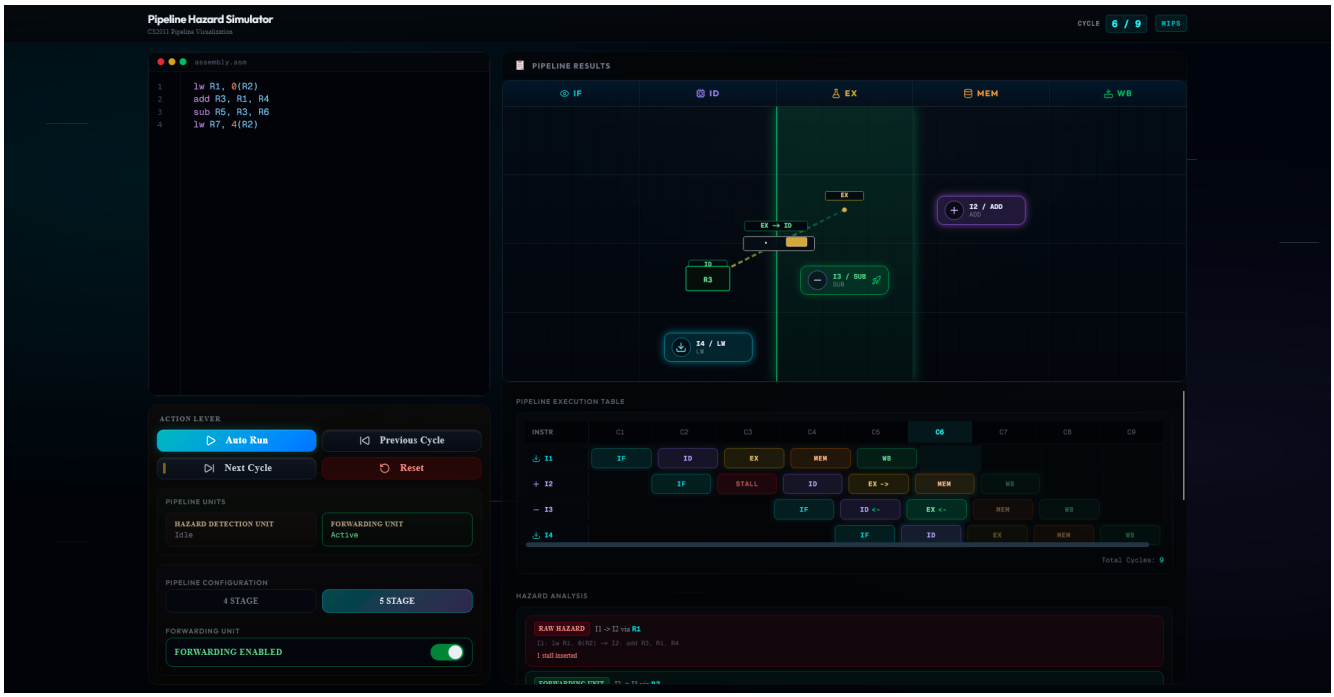

Figure 2 – 5-Stage pipeline, forwarding enabled (Cycle 6/9). Dashed green arrows show EX→ID and MEM→EX forwarding paths. Only 1 stall remains for the load-use pair I1→I2.

4.3 Test Case C – 4-Stage Pipeline, Forwarding Disabled

Instructions:

```
I1: add R0, R1, R2
I2: add R0, R0, R3
I3: sub R4, R0, R5
```

The 4-stage pipeline merges MEM and WB. I2 reads R0 written by I1 (RAW, 3 stalls), and I3 reads R0 written by I2 (RAW, 3 stalls).

Pipeline Execution Table:

INSTR	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
I1 (ADD)	IF	ID	EX	MEM/WB									
I2 (ADD)		IF	STALL	STALL	STALL	ID	EX	MEM/WB					

INSTR	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
I3 (SUB)			IF	STALL	STALL	STALL	IF	STALL	STALL	STALL	ID	EX	MEM/WB

Total Cycles: 13

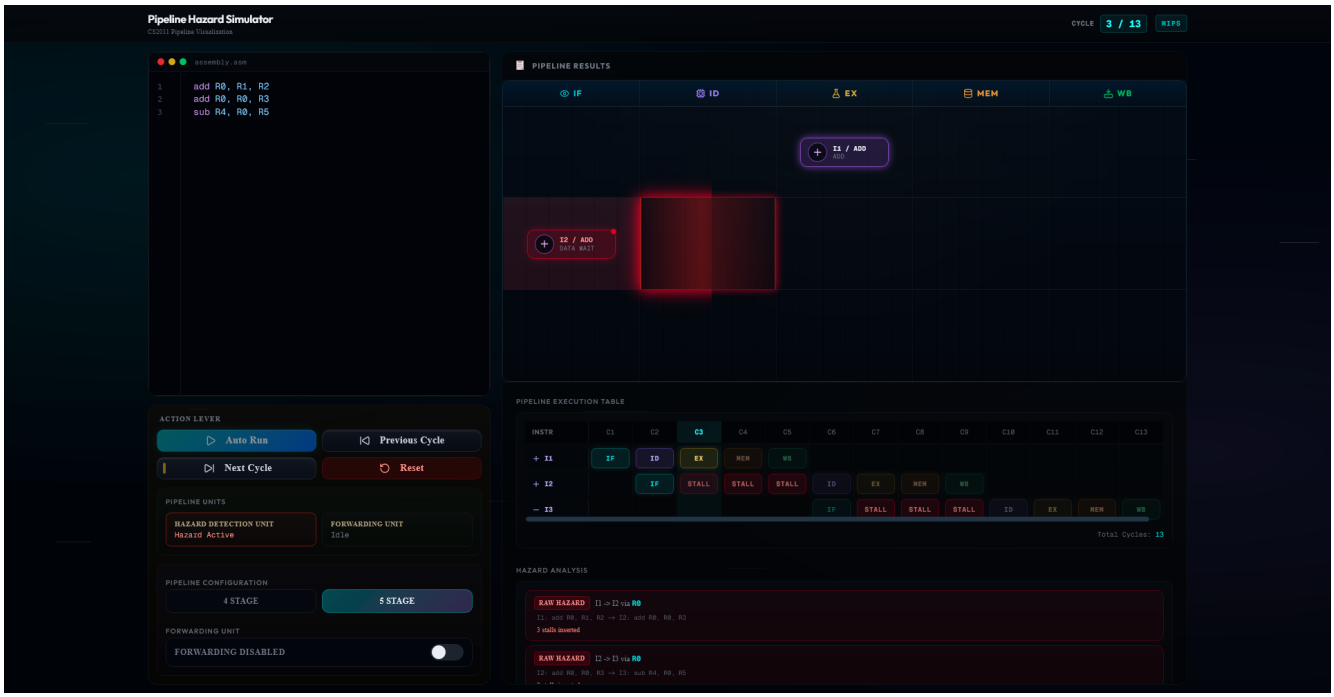

Figure 3 – 4-Stage pipeline, forwarding disabled (Cycle 3/13). Both RAW hazards listed in Hazard Analysis panel. Three STALL bubbles inserted after each dependent instruction.

4.4 Test Case D – 4-Stage Pipeline, Forwarding Enabled

Same sequence as Test Case C.

With forwarding active, EX→EX paths resolve both RAW hazards with no stalls.

Pipeline Execution Table:

INSTR	C1	C2	C3	C4	C5	C6	C7
I1 (ADD)	IF	ID	EX→	MEM/WB			
I2 (ADD)		IF	ID←	EX→	MEM/WB		
I3 (SUB)			IF	ID←	EX←	MEM/WB	

Total Cycles: 7 (down from 13)

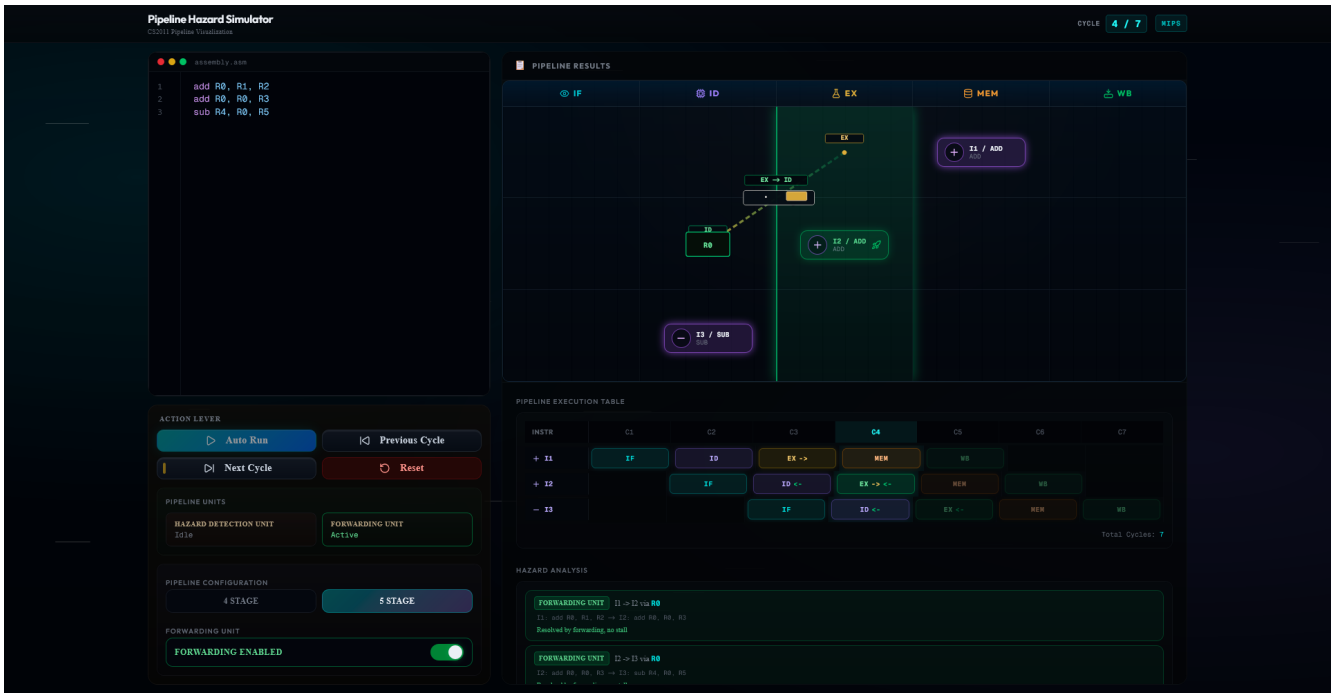

Figure 4 – 4-Stage pipeline, forwarding enabled (Cycle 4/7). Forwarding Unit shows Active. Hazard Analysis confirms resolution via forwarding, no stalls inserted.

---

5. Summary

Configuration	Forwarding Disabled	Forwarding Enabled	Stalls Saved
5-Stage (Test A/B)	14 cycles	9 cycles	5 cycles
4-Stage (Test C/D)	13 cycles	7 cycles	6 cycles

---

6. How to Run

Online (recommended):
Open https://pipeline-hazard-simulator.vercel.app/ in any modern browser. No installation required.

Local build:

```
# Unzip the submitted archive
unzip app.zip
cd app

# Install dependencies
npm install

# Start development server
npm run dev
```

The applet will be available at `http://localhost:3000`.

CS2011 – Foundations of Computer Systems | Plaksha University | Lab Assignment II